Date: 10-10-2026

CO-1 - AT-1

APPLICATION BASED QUESTIONS

COURES CODE: DSA0308

COUSRE NAME: NLP

Reg no: 192472313 **Name: Pavan Jana**

Section 1: Regular Expressions – Resume Information Extraction

Explanation:

The Resume Information Extraction system is a Python-based application that uses Regular Expressions to extract important details from resumes. **It identifies the candidate's name, email address, mobile number, technical skills, and years of experience.** The system checks whether the candidate has at least two years of experience and knows Python. If both conditions are satisfied, the candidate is considered eligible for shortlisting.

1. Uses the Python re module to extract the candidate's name, email, mobile number, technical skills, and years of experience.
2. Identifies skills such as Python, Java, SQL, Machine Learning, and NLP from the resume.
3. Checks whether the candidate has at least 2 years of experience and Python skills to determine eligibility.

Python Program

```
import re

resume = """

Name: Pavan Jana

Email: pavan@gmail.com

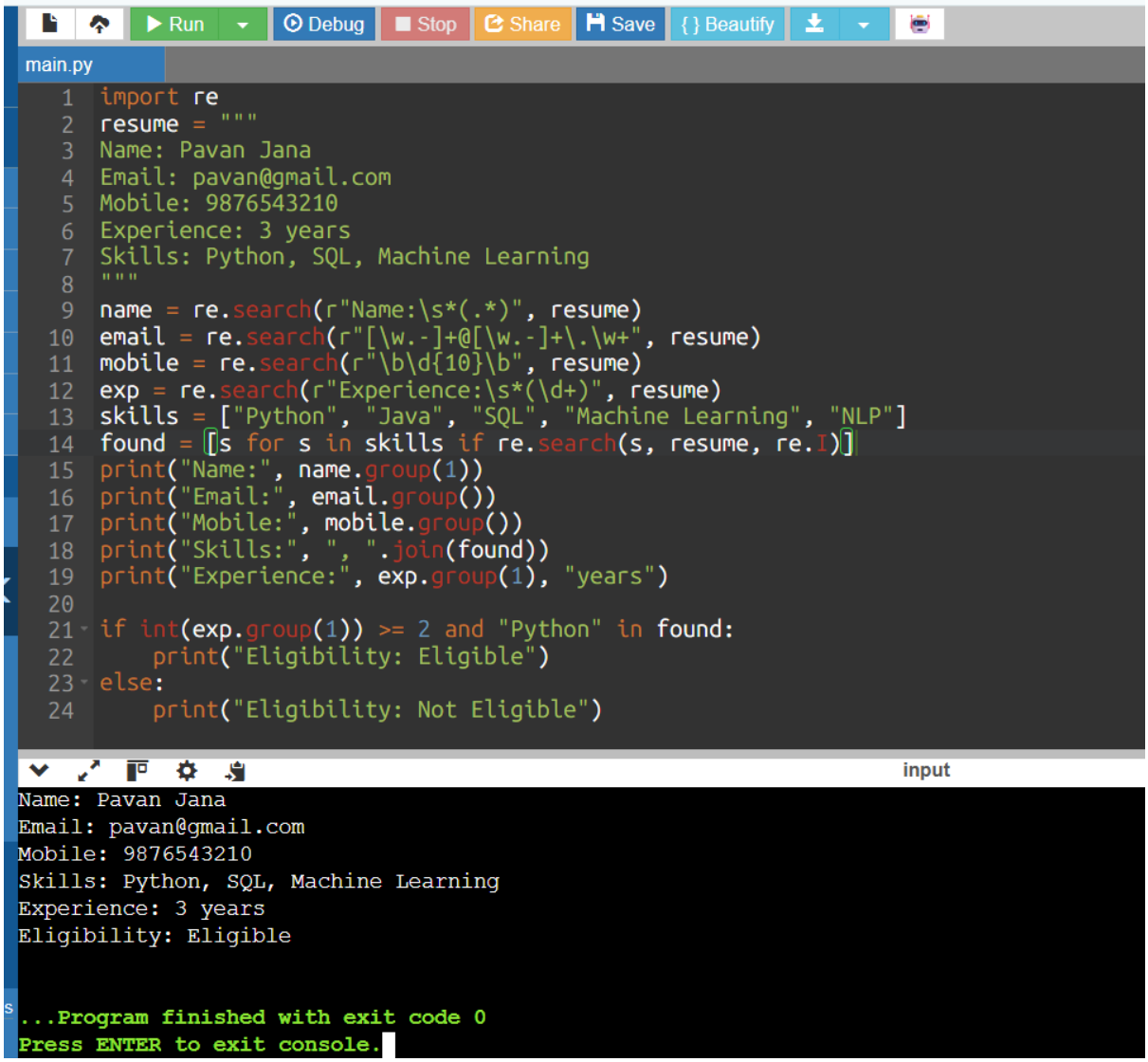
Mobile: 9876543210
```

```
Experience: 3 years
Skills: Python, SQL, Machine Learning
"""
name = re.search(r"Name:\s*(.*)", resume)
email = re.search(r"[\w.-]+@[\w.-]+\.\w+", resume)
mobile = re.search(r"\b\d{10}\b", resume)
exp = re.search(r"Experience:\s*(\d+)", resume)
skills = ["Python", "Java", "SQL", "Machine Learning", "NLP"]
found = [s for s in skills if re.search(s, resume, re.I)]
print("Name:", name.group(1))
print("Email:", email.group())
print("Mobile:", mobile.group())
print("Skills:", ", ".join(found))
print("Experience:", exp.group(1), "years")

if int(exp.group(1)) >= 2 and "Python" in found:
    print("Eligibility: Eligible")
else:
    print("Eligibility: Not Eligible")
```

Output

```
Name: Pavan Jana
Email: pavan@gmail.com
Mobile: 9876543210
Skills: Python, SQL, Machine Learning
Experience: 3 years
Eligibility: Eligible
```



Result

The program successfully extracts resume information and checks the candidate's eligibility based on experience and Python skills.

Section 2: Regular Expressions – Product Search System

Explanation:

The Product Search System uses Python Regular Expressions to search for products based on different keyword patterns. It supports exact keyword searches, prefix searches, suffix searches, partial keyword searches, and case-insensitive searches. The system displays the matching product names and the total number of products found for each search.

1. Uses Regular Expressions to search product names using exact, prefix, suffix, and partial keywords.
2. Supports case-insensitive searching, allowing users to find products regardless of uppercase or lowercase letters.
3. Displays all matching product names and the total number of products found for each search.

Python Program

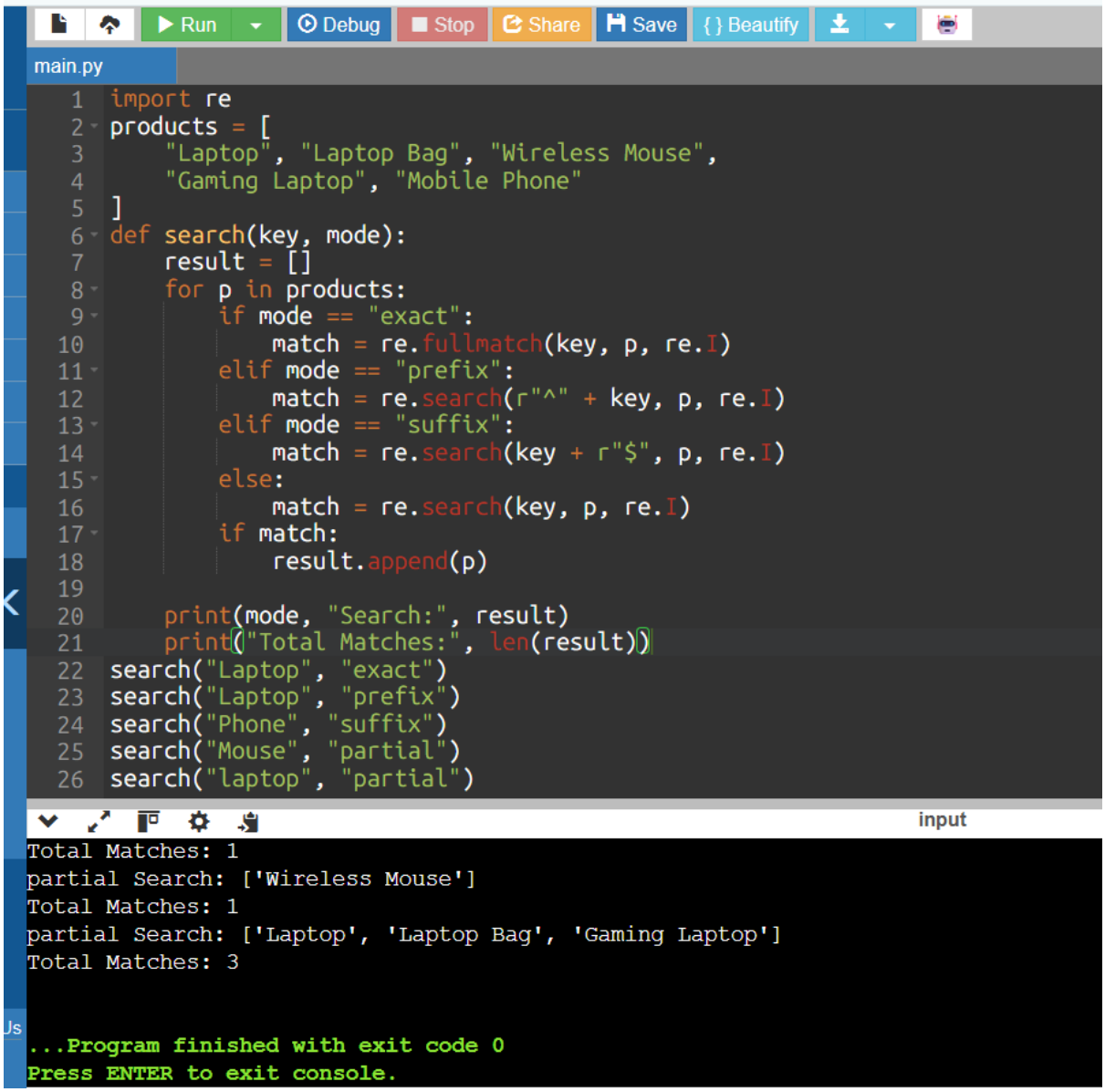
```
import re
products = [
    "Laptop", "Laptop Bag", "Wireless Mouse",
    "Gaming Laptop", "Mobile Phone"
]
def search(key, mode):
    result = []
    for p in products:
        if mode == "exact":
            match = re.fullmatch(key, p, re.I)
        elif mode == "prefix":
            match = re.search(r"^" + key, p, re.I)
        elif mode == "suffix":
            match = re.search(key + r"$", p, re.I)
        else:
```

```
            match = re.search(key, p, re.I)
        if match:
            result.append(p)

    print(mode, "Search:", result)
    print("Total Matches:", len(result))
search("Laptop", "exact")
search("Laptop", "prefix")
search("Phone", "suffix")
search("Mouse", "partial")
search("laptop", "partial")
```

Output

```
exact Search: ['Laptop']
Total Matches: 1
prefix Search: ['Laptop', 'Laptop Bag']
Total Matches: 2
suffix Search: ['Mobile Phone']
Total Matches: 1
partial Search: ['Wireless Mouse']
Total Matches: 1
partial Search: ['Laptop', 'Laptop Bag', 'Gaming Laptop']
Total Matches: 3`
```



Result

The program successfully performs different types of product searches and displays the matching products with their counts.

Section 3: Regular Expressions – University Registration System

Explanation:

The University Registration System is a Python-based application that uses Regular Expressions to validate student details before course enrollment. **It checks the student register number, institutional email address, course code, semester, and mobile number against predefined formats.** The program displays a validation message for each field and generates a final registration report. Registration is successful only when all fields are valid.

Python Program

```
import re
reg_no = "23CSE1234"
email = "student@university.edu"
course = "CSE101"
semester = "3"
mobile = "9876543210"
checks = {
    "Register Number": bool(re.fullmatch(r"\d{2}CSE\d{4}", reg_no)),
    "Institutional Email": bool(re.fullmatch(
        r"[\w.-]+@university\.edu", email)),
    "Course Code": bool(re.fullmatch(r"[A-Z]{3}\d{3}", course)),
    "Semester": bool(re.fullmatch(r"[1-8]", semester)),
    "Mobile Number": bool(re.fullmatch(r"[6-9]\d{9}", mobile))
}
for field, valid in checks.items():
    print(field + ":", "Valid" if valid else "Invalid")
if all(checks.values()):
    print("Registration Status: Successful")
else:
    print("Registration Status: Failed")
```

Output

Register Number: Valid

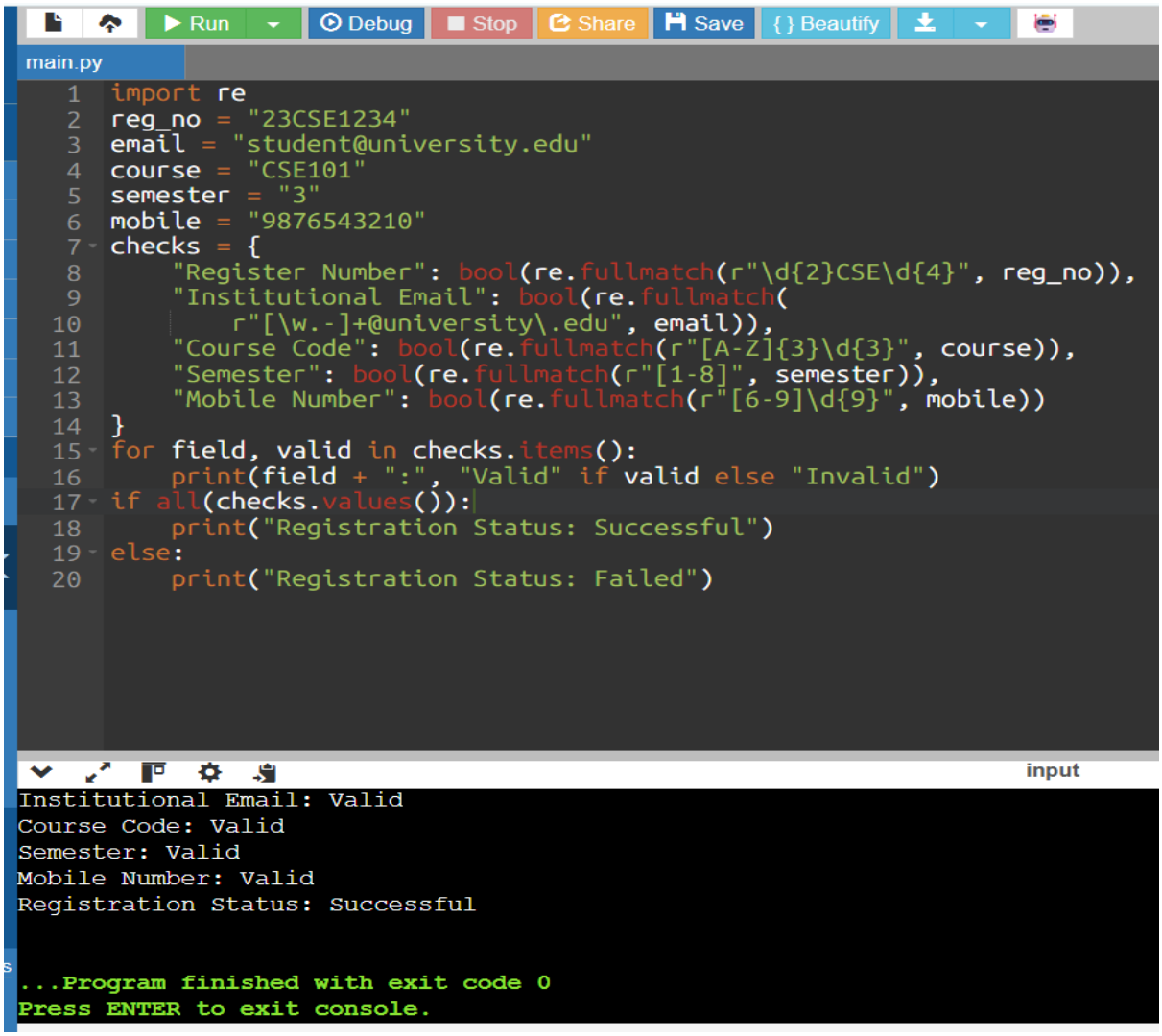
Institutional Email: Valid

Course Code: Valid

Semester: Valid

Mobile Number: Valid

Registration Status: Successful



Result

The program successfully validates the student's registration details and displays the final registration status.